Table of Contents

4. LQ Model Predictive Control With Constraints

4.1 LQ MPC with Constraints for Regulation

LQ-MPC problem with constraints

Define now a Constrained LQ Model Predictive Control (P-LQMPC) problem,

$$
\begin{aligned}
\underset{u_0,u_1,\cdots,u_{N-1}}{\text{Minimize}} \quad & J_N = \sum_{k=0}^{N-1} x_k^{\mathrm{T}} Q x_k + u_k^{\mathrm{T}} R u_k + x_N^{\mathrm{T}} P x_N \\
\text{subject to} \quad & x_{k+1} = Ax_k + Bu_k \\
& x_0 = \text{current state} \\
& x_{min} \leq x_k \leq x_{max}, \;\; k = 1, \cdots, N \\
& u_{min} \leq u_k \leq u_{max}, \;\; k = 0, \cdots, N-1
\end{aligned}
$$

The MPC feedback law is defined by the first optimal move, u_0^*, i.e, $u_{MPC}^*(x_0) = u_0^*$.

Remark 1: More general constraints that are affine in state and control and the case when constraint, control and prediction horizons are different ($N_c \neq N_u \neq N$) can be treated similarly.

Remark 2: Suppose $P = P_\infty$, where P_∞ is the solution to the corresponding DARE. If the constraints are inactive (hold as strict inequalities) for the optimal solution at x_0, the solutions to P-LQMPC and P-LQ coincide. This is typically the case near the origin, hence $u_{MPC}(x_0) = u_0^* = K_\infty x_0$ and local closed-loop stability is guaranteed.

Reduction to quadratic program

Stack the states and controls into vectors $X \in \mathbb{R}^{N \cdot n_x}$ and $U \in \mathbb{R}^{N \cdot n_u}$,

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \quad U = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix}.$$

We will now demonstrate that P-LQMPC reduces to a quadratic programming problem, that is a problem of minimizing a quadratic function subject to linear constraints.

Stacked states vector is expressed as a linear function of stacked controls vector

The state transition formula,

$$x_k = A^k x_0 + \sum_{i=0}^{k-1} A^{k-1-i} B u_i,$$

yields the following relation,

$$X = SU + Mx_0,$$

where

$$S = \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ A^{N-1}B & A^{N-2}B & \cdots & B \end{bmatrix}, \quad M = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix}.$$

Constraints on stacked up states vector

Let

$$X_{min} = \begin{bmatrix} x_{min} \\ x_{min} \\ \vdots \\ x_{min} \end{bmatrix}, \quad X_{max} = \begin{bmatrix} x_{max} \\ x_{max} \\ \vdots \\ x_{max} \end{bmatrix}.$$

Then, the state constraints have the form,

$$X_{min} \leq X = SU + Mx_0 \leq X_{max},$$

which can be written as

$$\begin{bmatrix} SU \\ -SU \end{bmatrix} \leq \begin{bmatrix} X_{max} - Mx_0 \\ -X_{min} + Mx_0 \end{bmatrix},$$

or

$$\begin{bmatrix} S \\ -S \end{bmatrix} U \leq \begin{bmatrix} X_{max} - Mx_0 \\ -X_{min} + Mx_0 \end{bmatrix}.$$

Augmenting control constraints

Let

$$U_{min} = \begin{bmatrix} u_{min} \\ u_{min} \\ \vdots \\ u_{min} \end{bmatrix}, \quad U_{max} = \begin{bmatrix} u_{max} \\ u_{max} \\ \vdots \\ u_{max} \end{bmatrix}.$$

Then, the constraints on the stacked-up controls vector have the form,

$$U_{min} \leq U \leq U_{max},$$

which can be written as

$$\begin{bmatrix} I \\ -I \end{bmatrix} U \leq \begin{bmatrix} U_{max} \\ -U_{min} \end{bmatrix}.$$

State and control constraints

The state and control constraints thus take the form,

$$\begin{bmatrix} S \\ -S \\ I \\ -I \end{bmatrix} U \leq \begin{bmatrix} X_{max} - Mx_0 \\ -X_{min} + Mx_0 \\ U_{max} \\ -U_{min} \end{bmatrix}.$$

This can be written as

$$GU \leq W + Tx_0,$$

$$G = \begin{bmatrix} S \\ -S \\ I \\ -I \end{bmatrix}, \quad W = \begin{bmatrix} X_{max} \\ -X_{min} \\ U_{max} \\ -U_{min} \end{bmatrix}, \quad T = \begin{bmatrix} -M \\ M \\ 0 \\ 0 \end{bmatrix}$$

Cost function transformation

Let

$$\bar{Q} = \begin{bmatrix} Q & \cdots & & \cdots & 0 \\ 0 & Q & & & \\ \vdots & & \ddots & & \\ 0 & \cdots & & Q & \\ 0 & \cdots & & 0 & P \end{bmatrix}, \quad \bar{R} = \begin{bmatrix} R & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & R \end{bmatrix},$$

Then

$$\begin{aligned} J_N &= \sum_{k=0}^{N-1} x_k^{\mathrm{T}} Q x_k + u_k^{\mathrm{T}} R u_k + x_N^{\mathrm{T}} P x_N \\ &= X^{\mathrm{T}} \bar{Q} X + U^{\mathrm{T}} \bar{R} U + x_0^{\mathrm{T}} Q x_0 \\ &= (SU + M x_0)^{\mathrm{T}} \bar{Q} (SU + M x_0) + U^{\mathrm{T}} \bar{R} U + x_0^{\mathrm{T}} Q x_0 \\ &= U^{\mathrm{T}} (S^{\mathrm{T}} \bar{Q} S + \bar{R}) U + 2 x_0^{\mathrm{T}} M^{\mathrm{T}} \bar{Q} S U + x_0^{\mathrm{T}} M^{\mathrm{T}} \bar{Q} M x_0 + x_0^{\mathrm{T}} Q x_0 \\ &= U^{\mathrm{T}} H U + 2 q^{\mathrm{T}} U + c \end{aligned}$$

where $H = S^{\mathrm{T}} \bar{Q} S + \bar{R}$, $q = S^{\mathrm{T}} \bar{Q} M x_0$, $c = x_0^{\mathrm{T}} (Q + M^{\mathrm{T}} \bar{Q} M) x_0$.

Reduction to quadratic program (QP)

Thus we have demonstrated that P-LQMPC reduces to a quadratic programming problem

$$\begin{aligned} \underset{U \in \mathbb{R}^{N \times n_u}}{\text{Minimize}} \quad & J_N = U^{\mathrm{T}} H U + 2 q^{\mathrm{T}} U \\ \text{subject to} \quad & G U \leq W + T x_0. \\ & (q = S^{\mathrm{T}} \bar{Q} M x_0) \end{aligned}$$

Since $H = (S^{\mathrm{T}} \bar{Q} S + \bar{R}) = H^{\mathrm{T}} \succ 0$, the cost is a strictly convex function of U, and the above optimization problem is a strictly convex QP with affine inequality constraints. We will discuss convexity and strict convexity a bit later. For now, just note these are properties that ensure uniqueness of the solution if one exists and convergence of numerical algorithms for finding it.

Note that the optimization problem is parameterized by the current state x_0.

Note that we dropped c from J_N (constant does not matter in minimization)

MPC Feedback Law

Once the optimization problem is solved, and the solution $U^*(x_0)$ is obtained, the LQ-MPC feedback law is defined by the first move/first element of the control sequence as

$$u_{MPC}(x_0) = \begin{bmatrix} \mathbb{I}_{n_u \times n_u} & 0 & \cdots & 0 \end{bmatrix} U^*(x_0)$$

QP solver is needed in the constrained case

What do you have to specify to a solver?

$$\underset{U \in \mathbb{R}^{N \times n_u}}{\text{Minimize}} \quad J_N = \frac{1}{2} U^{\mathrm{T}} H U + q^{\mathrm{T}} U$$

$$\text{subject to} \quad GU \leq \tilde{W} \quad (\tilde{W} = W + T x_0). \qquad (q = S^{\mathrm{T}} \bar{Q} M x_0)$$

solveQP(H,q,G,$\tilde{W}$,opts)

Common optional parameters (opts) are

- Bounds for the variables (min and max)
- Initial solution guess (otherwise the solver finds one)
- Maximum number of iterations (or max time)
- Termination condition (ϵ, $||J_N(U) - J_N(U^*)|| \leq \epsilon$, dual variables provide bound on the cost deviation from optimal - to be discussed)
- Constraint acceptance (ϵ, $GU - \tilde{W} \leq \mathbf{1}\epsilon$)

Matlab: `quadprog.m`

4.2 Design of LQ-MPC with Constraints for Tracking

Tracking LQ-MPC design

MPC controllers can be designed to perform set-point tracking, i.e., cause a given output y of the system,

$$x_{k+1} = Ax_k + Bu_k, \quad y_k = Cx_k,$$

follow a specified command (set-point) r. Specifically, if $r \equiv const$ the control design objective is to achieve convergence of the output to the command, i.e.,

$$y_k \to r \text{ as } k \to \infty \ .$$

Remark 1: The ability to achieve zero offset in the output versus a constant command in steady-state despite potential model mismatch or disturbances is known as zero offset or offset free tracking property. To achieve this $n_u \geq n_y$ (more inputs than outputs) is typically required.

Remark 2: Controllers well-designed for constant set-point tracking often perform well and are practically used even when command r is slowly varying.

Tracking LQ-MPC design

Remark 3: If $r \neq 0$, while $e_k \to 0$, we cannot in general expect that $x_k \to 0$ and $u_k \to 0$ as $k \to \infty$. Hence the cost function needs to be somehow changed to penalize the right quantities.

4.3 Reference Tracking Formulation 1

Reference tracking formulation 1

To achieve reference tracking, we could consider the following modified cost function,

$$J_N = \sum_{k=0}^{N-1} (x_k - x_s)^{\mathrm{T}} Q (x_k - x_{\mathrm{s}}) + (u_k - u_{\mathrm{s}})^{\mathrm{T}} R (u_k - u_{\mathrm{s}}),$$

where $x_{\mathrm{s}} = x_{\mathrm{s}}(r)$ and $u_{\mathrm{s}} = u_{\mathrm{s}}(r)$ are the equilibrium state-control pair corresponding to the chosen value of output, $y = r$.

The values have to satisfy

$$\begin{aligned} A x_s + B u_s &= x_s \\ C x_s &= r \end{aligned}$$

or

$$\begin{bmatrix} A - I & B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix}.$$

$$\Rightarrow x_s(r), u_s(r)$$

For instance, if $(C(I-A)^{-1}B)$ is invertible, then $x_s(r) = (I-A)^{-1}B u_s(r),\ u_s(r) = (C(I-A)^{-1}B)^{-1} r$

Offset-free tracking

This formulation does not guarantee offset-free tracking.

The issue is that the assumed prediction model,

$$x_{k+1} = Ax_k + Bu_k, \; y_k = Cx_k$$

may not match the actual system dynamics,

$$x_{k+1} = Ax_k + Bu_k + B_d d_k, \; y_k = Cx_k + D_d d_k,$$

where d_k represents a model mismatch or a disturbance.

The model mismatch will likely cause the mismatch between the output and constant reference command in steady-state.

To ensure zero steady-state error, a common technique in classical control is the use of integral action.

Reference tracking formulation 1

To achieve reference tracking, we could consider the following modified cost function,

$$J_N = \sum_{k=0}^{N-1} (x_k - x_{\mathrm{s}}(v))^{\mathrm{T}} Q (x_k - x_{\mathrm{s}}(v)) + (u_k - u_{\mathrm{s}}(v))^{\mathrm{T}} R (u_k - u_{\mathrm{s}}(v)),$$

with v not necessarily equal to r.

We could then add an extra integral control loop outside of MPC controller as

$$v_{k+1} = v_k + K_{\mathrm{int}}(y_k - r)$$

where K_{int} is a gain matrix, to help ensure zero steady-state offset (if feasible under constraints) in presence of disturbances/uncertainties.

From computational perspective this approach to achieving tracking has relatively low computational complexity (other approaches will require model augmentation with extra states), however, an outside integrator has to be used to achieve zero steady-state offset and hence care is needed to avoid integrator windup. The transient response may also be degraded compared to other approaches but this is problem-dependent.

4.4 Reference Tracking Formulation 2

Reference tracking formulation 2

Another possibility is to consider the following cost function,

$$J_N = \sum_{k=0}^{N-1} e_k^{\mathrm{T}} Q_e e_k + \Delta u_k^{\mathrm{T}} R \Delta u_k,$$

where $\Delta u_k = u_k - u_{k-1}$, $e_k = y_k - r = Cx_k - r$, and $Q_e^{\mathrm{T}} = Q_e \in \mathbb{R}^{n_e \times n_e}$, $R^{\mathrm{T}} = R \in \mathbb{R}^{n_u \times n_u}$, $Q_e \succeq 0$, $R \succ 0$.

Remark 1: Note that we penalize the control increments, Δu_k, rather than actual controls, u_k, since to maintain an output at a nonzero commanded value in steady-state, a non-zero control is (typically) required and driving control to zero does not make sense.

Remark 2: When tuning R, larger values will cause less aggressive control (slower varying) rather than smaller valued if u_k were penalized.

Question: How can we reduce this problem to standard LQ-MPC formulation?

Reference tracking formulation 2

Consider the objective to ensure output command tracking, i.e.,

$$y_k = Cx_k \to r.$$

Define an augmented system with the extended state $x_k^{\text{ext}} = [x_k^{\text{T}} \; u_{k-1}^{\text{T}} \; r_k^{\text{T}}]^{\text{T}}$ given by

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k \\ u_k &= u_{k-1} + \Delta u_k \\ r_{k+1} &= r_k \end{aligned}$$

or

$$x_{k+1}^{\text{ext}} = \begin{bmatrix} A & B & 0 \\ 0 & \mathbb{I}_{n_u \times n_u} & 0 \\ 0 & 0 & \mathbb{I}_{n_r \times n_r} \end{bmatrix} x_k^{\text{ext}} + \begin{bmatrix} B \\ \mathbb{I}_{n_u \times n_u} \\ 0 \end{bmatrix} \Delta u_k$$

For the augmented system, we can define the tracking error as the output,

$$e_k = y_k - r_k = \begin{bmatrix} C & 0 & -\mathbb{I}_{n_r \times n_r} \end{bmatrix} x_k^{\text{ext}} = E x_k^{\text{ext}}.$$

Reference tracking formulation 2

The MPC feedback law can be defined based on the following optimal control problem,

$$\underset{\Delta u_0, \Delta u_1, \cdots, \Delta u_{N-1}}{\text{Minimize}} \quad J_N = \sum_{k=0}^{N-1} e_k^{\mathrm{T}} Q_e e_k + \Delta u_k^{\mathrm{T}} R \Delta u_k$$

$$\text{subject to} \quad x_{k+1}^{\text{ext}} = \begin{bmatrix} A & B & 0 \\ 0 & \mathbb{I}_{n_u \times n_u} & 0 \\ 0 & 0 & \mathbb{I}_{n_r \times n_r} \end{bmatrix} x_k^{\text{ext}} + \begin{bmatrix} B \\ \mathbb{I}_{n_u \times n_u} \\ 0 \end{bmatrix} \Delta u_k$$

$$e_k = E x_k^{\text{ext}}$$

$$x_0^{\text{ext}} = \begin{bmatrix} x_0^{\mathrm{T}} & u_{-1}^{\mathrm{T}} & r^{\mathrm{T}} \end{bmatrix}^{\mathrm{T}}, \; x_0 = \text{current state}$$

$$r_k = r, \;\; k = 0, \cdots, N$$

$$x_{min} \leq x_k \leq x_{max}, \;\; k = 1, \cdots, N$$

$$u_{min} \leq u_k \leq u_{max}, \;\; k = 0, \cdots, N-1$$

Note: $e_k^{\mathrm{T}} Q_e e_k = (x_k^{\text{ext}})^{\mathrm{T}} E^{\mathrm{T}} Q_e E x_k^{\text{ext}} \Rightarrow Q = E^{\mathrm{T}} Q_e E$

$$\Rightarrow J_N = \sum_{k=0}^{N-1} (x_k^{\text{ext}})^{\mathrm{T}} Q x_k^{\text{ext}} + \Delta u_k^{\mathrm{T}} R \Delta u_k$$

Reference tracking formulation 2

The MPC feedback law is defined by the first optimal move, Δu_0^*, i.e,

$$\Delta u_{MPC}^*(x_0, u_{-1}, r) = \Delta u_0^*.$$

Suppose x_t is the current state, u_{t-1} is the previous control and r_t is the current reference command during the operation. Then the control action is computed according to

$$u_t = u_{t-1} + \Delta u_{MPC}^*(x_t, u_{t-1}, r_t).$$

Reference tracking formulation 2

Unfortunately, reference tracking formulation 2 by itself will not guarantee offset-free tracking in presence of model mismatch even when there are no constraints.

Consider an example:

$$x_{k+1} = \alpha x_k + \beta u_k$$

$$y_k = x_k$$

$$J = q(y_1 - r)^2 + \Delta u_0^2 \to \min_{\Delta u_0}$$

$$\Delta u_0 = u_0 - u_{-1}$$

$x_0 =$ current state, $u_0 =$ current control,

$u_{-1} =$ previous control

There are no constraints, and the solution can be worked out by hand or symbolically

```
a = sym('a'); b = sym('b');
q = sym('q'); um = sym('um');
r = sym('r'); x0 = sym('x0');
u = sym('u');
J = q*(a*x0+b*u-r)^2+(u-um)^2;
simplify(solve(diff(J,u),u))
```

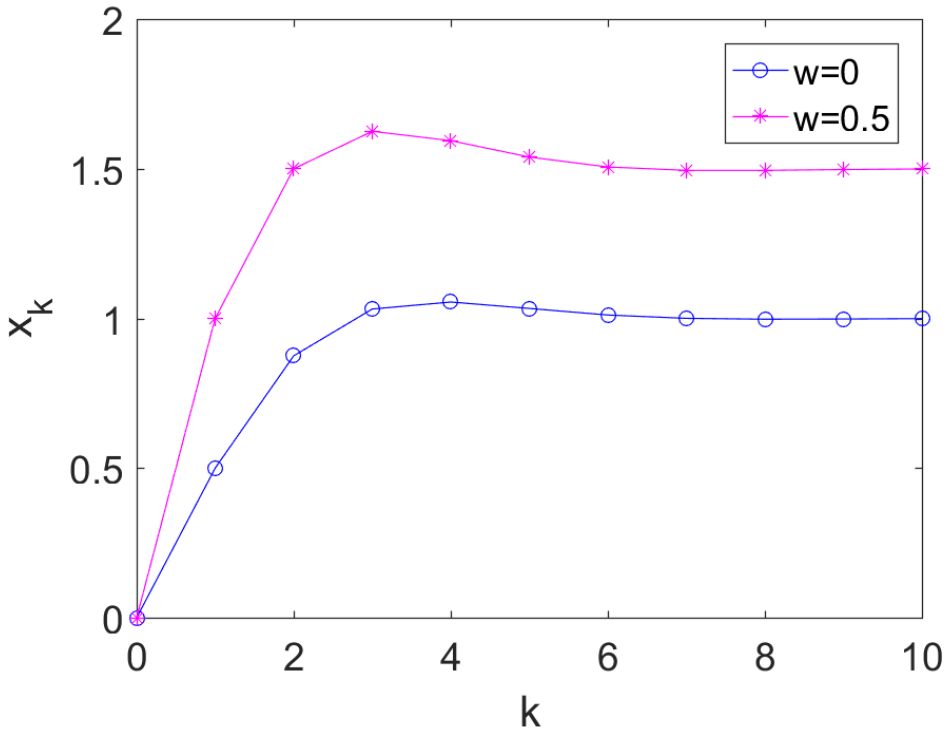
$$\Rightarrow \quad u_0^* = \frac{u_{-1} + \beta q(r - \alpha x_0)}{q\beta^2 + 1}$$

Reference tracking formulation 2

Simulate closed-loop with a constant disturbance:

$$y_{t+1} = \alpha y_t + \beta u_t + d_t, \;\; u_t = \frac{u_{t-1} + \beta q(r - \alpha y_t)}{q\beta^2 + 1}$$

$$\alpha = \frac{1}{2}, \;\; \beta = 1, \;\; r = 1, \;\; q = 1, \;\; d_t = w$$



Longer horizons have similar issue!

4.5 Reference Tracking Formulation 2 with Disturbance Observer

Reference tracking formulation 2

One approach to obtain offset-free tracking is to use a disturbance observer

Consider the model:

$$x_{t+1} = Ax_t + Bu_t + B_d d_t$$
$$y_t = Cx_t$$

and treat the disturbance d_t as constant in time, i.e.,

$$d_{t+1} = d_t.$$

Design an observer to estimate x_t and d_t:

$$\hat{x}_{t+1} = A\hat{x}_t + Bu_t + B_d\hat{d}_t + L_1(y_t - C\hat{x}_t)$$
$$\hat{d}_{t+1} = \hat{d}_t + L_2(y_t - C\hat{x}_t)$$

where $L = [L_1, \quad L_2]^{\mathrm{T}}$ is the observer gain which can be determined using discrete Linear Quadratic Estimation (Kalman filter) (`dlqe.m`)

Now include the estimated disturbance in the prediction model to let MPC controller know about it

Reference tracking formulation 2

The prediction model has the form:

$$\begin{aligned} x_{k+1|t} &= Ax_{k|t} + Bu_{k|t} + B_d d_{k|t} \\ d_{k+1|t} &= d_{k|t}, \\ y_{k|t} &= Cx_{k|t}, \\ x_{0|t} &= \hat{x}_t \text{ (given)}, d_{0|t} = \hat{d}_t \text{ (given)} \end{aligned}$$

Define the extended state vector as $z_{k|t} = [x_{k|t}^{\mathrm{T}}, d_{k|t}^{\mathrm{T}}]^{\mathrm{T}}$ and define the dynamics for the extended state:

$$z_{k+1|t} = \begin{bmatrix} A & B_d \\ 0 & \mathbb{I}_{n_d \times n_d} \end{bmatrix} z_{k|t} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_{k|t}$$

$$y_{k|t} = \begin{bmatrix} C & 0 \end{bmatrix} z_{k|t}$$

Now the standard LQ-MPC problem can be formulated with the cost

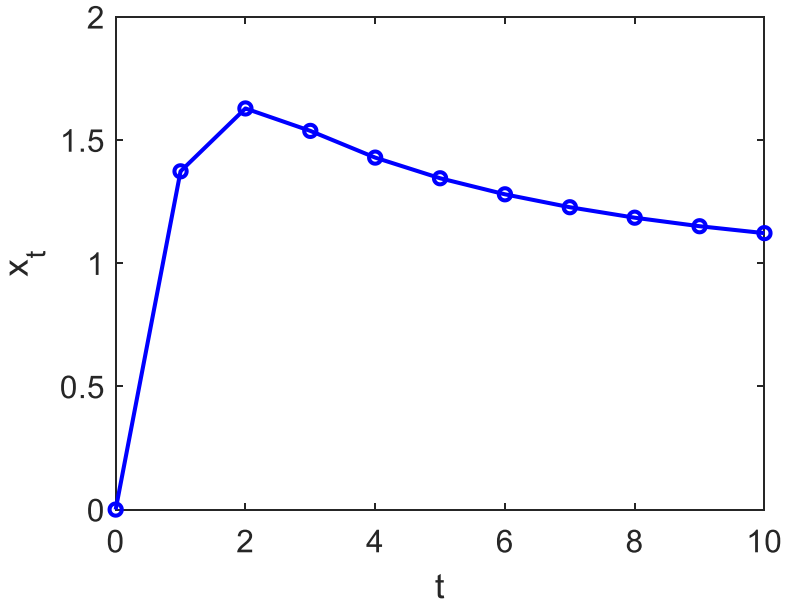
$$J_N = \sum_{k=0}^{N-1} e_{k|t}^{\mathrm{T}} Q_e e_{k|t} + \Delta u_{k|t}^{\mathrm{T}} R \Delta u_{k|t} = \sum_{k=0}^{N-1} (y_{k|t} - r)^{\mathrm{T}} Q_e (y_{k|t} - r) + \Delta u_{k|t}^{\mathrm{T}} R \Delta u_{k|t}$$

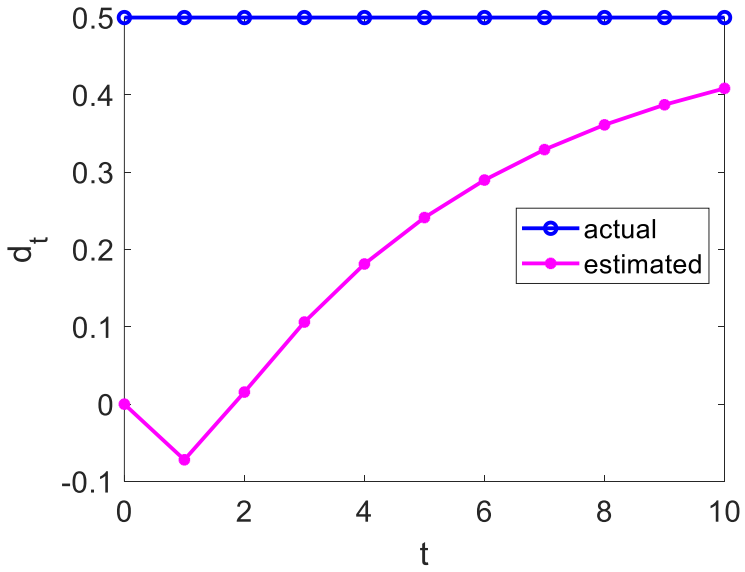
Reference tracking formulation 2

Back to our example...

$$A = \alpha = 0.5,\ B = \beta = 1,\ C = 1,\ B_d = B,\ L_1 = 0.6406,\ L_2 = 0.1896$$

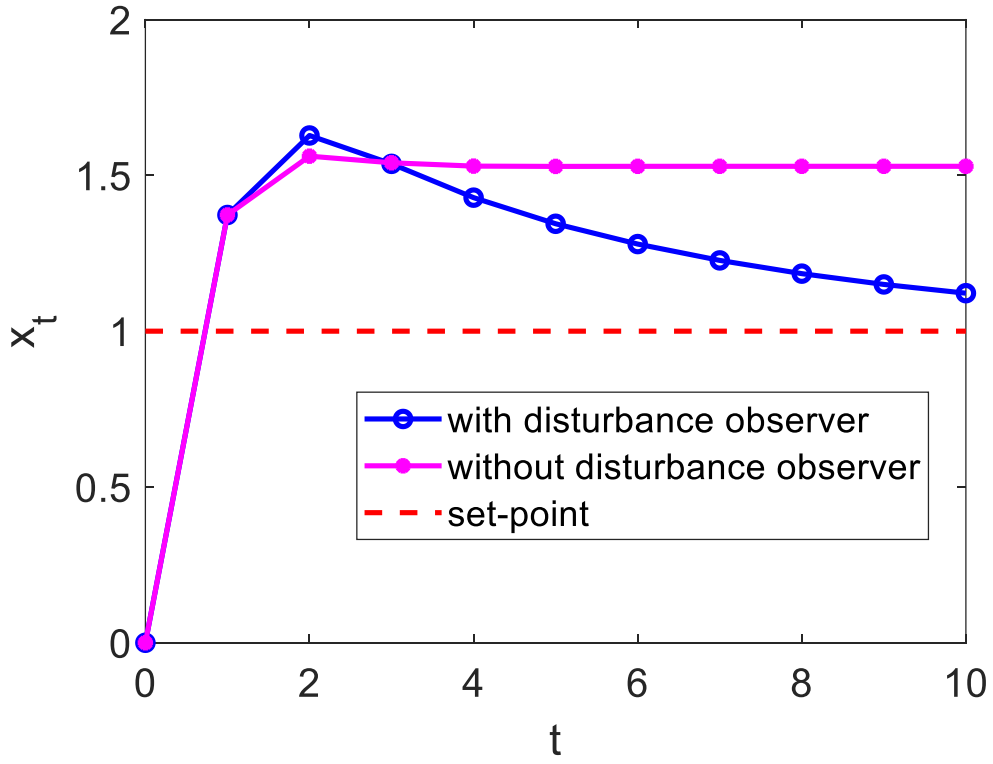
$$q = 1,\ R = 0.1,\ N = 4,\ r = 1 \text{ (set-point)}$$





Reference tracking formulation 2

Back to our example...



When is augmented system observable?

Consider the augmented system:

$$x_{t+1} = Ax_t + Bu_t + B_d d_t$$
$$d_{t+1} = d_t$$
$$y_t = Cx_t + C_d d_t$$

This is a bit more general due to presence of C_d

To be able to use disturbance input observer, the augmented system must be observable

Fact: Augmented system is observable if and only if the pair (C, A) is observable and the matrix

$$\begin{bmatrix} A - I & B_d \\ C & C_d \end{bmatrix}$$

has full column rank. (How can you prove that?)

The observer will have the form:

$$\hat{x}_{t+1} = A\hat{x}_t + Bu_t + B_d\hat{d}_t + L_1(y_t - C\hat{x}_t)$$
$$\hat{d}_{t+1} = \hat{d}_t + L_2(y_t - C\hat{x}_t - C_d\hat{d}_t)$$

4.6 Reference Tracking Formulation 3

Reference tracking formulation 3

It is possible to add an integrator explicitly:

$$x_{k+1} = Ax_k + Bu_k$$
$$i_{k+1} = i_k + (Cx_k - r)$$

Basic idea: If closed-loop is stable and reaches steady-state, then $i_k \to$ const implies $Cx_t \to r$ (irrespective of model mismatch or disturbances).

OCP problem is formulated as

$$J = \sum_{k=0}^{N-1} e_k^{\mathrm{T}} Q_e e_k + u_k^{\mathrm{T}} R u_k + i_k^{\mathrm{T}} Q_i i_k \to \min_{u_0, \cdots, u_{N-1}}$$
$$Q_e \succeq 0, \; R \succ 0, \; Q_i \succeq 0$$

Need to put it in the standard LQ-MPC form

Reference tracking formulation 3

$$\underbrace{\begin{bmatrix} x_{k+1} \\ i_{k+1} \\ r_{k+1} \end{bmatrix}}_{x_{k+1}^{\text{ext}}} = \begin{bmatrix} A & 0 & 0 \\ C & \mathbb{I}_{r\times r} & -\mathbb{I}_{r\times r} \\ 0 & 0 & \mathbb{I}_{r\times r} \end{bmatrix} \underbrace{\begin{bmatrix} x_k \\ i_k \\ r_k \end{bmatrix}}_{x_k^{\text{ext}}} + \begin{bmatrix} B \\ 0 \\ 0 \end{bmatrix} u_k$$

$$e_k = Cx_k - r = \underbrace{\begin{bmatrix} C & 0 & -\mathbb{I}_{r\times r} \end{bmatrix}}_{E} x_k^{\text{ext}}$$

$$i_k = \underbrace{\begin{bmatrix} 0 & \mathbb{I}_{r\times r} & 0 \end{bmatrix}}_{L} x_k^{\text{ext}}$$

$$J = \sum_{k=0}^{N-1} (x_k^{\text{ext}})^{\mathrm{T}} \underbrace{(E^{\mathrm{T}} Q_e E + L^{\mathrm{T}} Q_i L)}_{Q} x_k^{\text{ext}} + u_k^{\mathrm{T}} R u_k \to \min_{u_0,\cdots,u_{N-1}}$$

Reference tracking formulation 3

Controller: $u_{\text{MPC}}(x_0, i_0, r_0) = u_0^*$

If t is the current time, $u_t = u_{MPC}(x_t, i_t, r_t),\ i_{t+1} = i_t + (Cx_t - r)$

Note that the controller is dynamic, includes an integrator

The MPC control law is a function of the extended state which includes the actual state, x_t, the integrator state, i_t, and the reference value, r_t. Based on our previous results, in the unconstrained case this function is linear in these parameters!

If closed-loop is stable and the system converges to the steady-state, then $i_t \to$ const, $Cx_t \to r$ (irrespective of model mismatch or constant disturbance). It is not clear how to guarantee stability a priori, but it may be possible to check it a posteriori, e.g., using eigenvalue check if there are no constraints.

Note that i needs to be penalized in the cost, otherwise it is useless

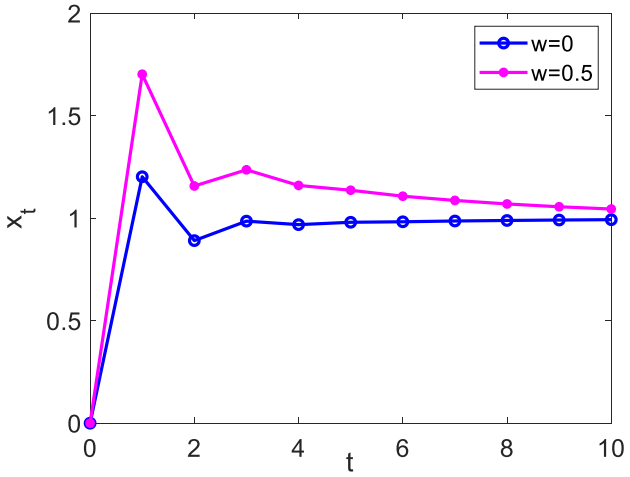
Reference tracking formulation 3

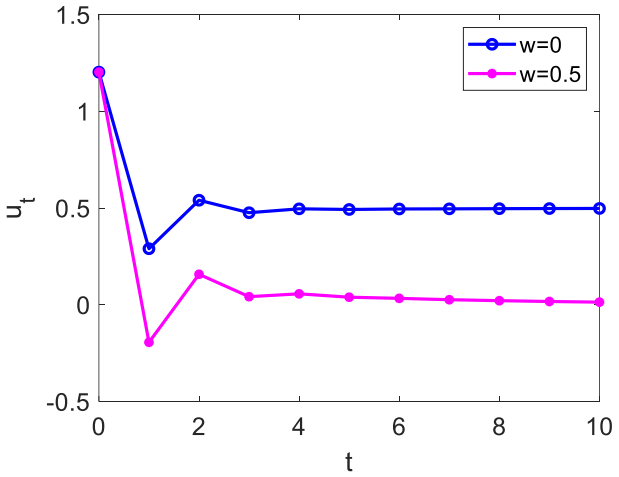
Back to the previous example...

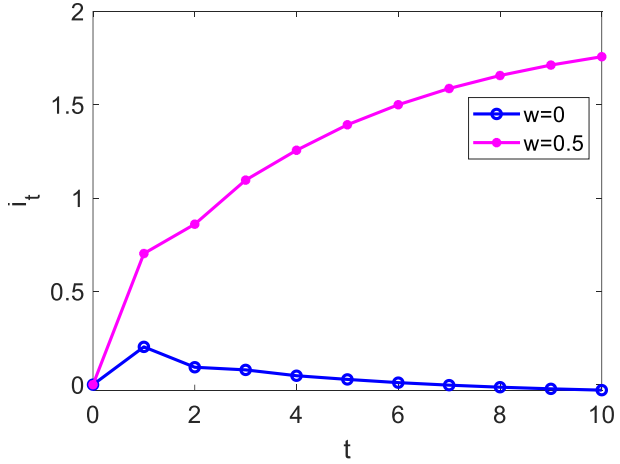
$$A = \alpha = 0.5,\ B = \beta = 1.0,\ C = 1,\ r = 1,\ Q_e = 1,\ Q_i = 0.2,\ R = 0.1$$

$N = 4$ (for $N = 2$ closed-loop is unstable and the integrator state grows unbounded)

Simulate: $x_{t+1} = Ax_t + Bu_{MPC}(x_t, i_t, r_t) + d_t,\ r_t = 1, i_{t+1} = i_t + (x - r),\ d_t = w$







Note that tracking of a constant command occurs with zero offset in steady-state!

Reference tracking formulation 3

Some questions:

- Why should i_k and u_k be penalized if they are not supposed to go to zero ?
- How can closed-loop stability be guaranteed?
- Boundness of the cost as $N \to \infty$ cannot be guaranteed. The formulation is not consistent with the infinite horizon LQR. Are we able to increase the horizon to get stable closed-loop?
- Not clear how to add the terminal penalty to be able to use shorter horizons.

Reference tracking formulation 3

In principle, we could change the cost to

$$J = \sum_{k=0}^{N-1} e_k^{\mathrm{T}} Q_e e_k + \Delta u_k^{\mathrm{T}} R \Delta u_k + i_k^{\mathrm{T}} Q_i i_k \rightarrow \min_{\Delta u_0, \cdots, \Delta u_{N-1}}$$

$$Q_e \succeq 0, \; R \succ 0, \; Q_i \succeq 0$$

This is a hybrid formulation between reference tracking formulations 2 and 3

Reference tracking formulation 3

Let's put this in the standard form:

$$\underbrace{\begin{bmatrix} x_{k+1} \\ i_{k+1} \\ u_k \\ r_{k+1} \end{bmatrix}}_{x_{k+1}^{\text{ext}}} = \begin{bmatrix} A & 0 & B & 0 \\ C & \mathbb{I}_{n_r \times n_r} & 0 & -\mathbb{I}_{n_r \times n_r} \\ 0 & 0 & \mathbb{I}_{n_u \times n_u} & 0 \\ 0 & 0 & 0 & \mathbb{I}_{n_r \times n_r} \end{bmatrix} \underbrace{\begin{bmatrix} x_k \\ i_k \\ u_{k-1} \\ r_k \end{bmatrix}}_{x_k^{\text{ext}}} + \begin{bmatrix} B \\ 0 \\ \mathbb{I}_{n_u \times n_u} \\ 0 \end{bmatrix} \Delta u_k$$

$$e_k = Cx_k - r = \underbrace{\begin{bmatrix} C & 0 & 0 & -\mathbb{I}_{n_r \times n_r} \end{bmatrix}}_{E} x_k^{\text{ext}}$$

$$i_k = \underbrace{\begin{bmatrix} 0 & \mathbb{I}_{n_r \times n_r} & 0 & 0 \end{bmatrix}}_{L} x_k^{\text{ext}}$$

$$J = \sum_{k=0}^{N-1} (x_k^{\text{ext}})^{\mathrm{T}} \underbrace{(E^{\mathrm{T}} Q_e E + L^{\mathrm{T}} Q_i L)}_{Q} x_k^{\text{ext}} + \Delta u_k^{\mathrm{T}} R \Delta u_k \rightarrow \min_{\Delta u_0, \cdots, \Delta u_{N-1}}$$

Reference tracking formulation 3

Controller: $u_{\mathrm{MPC}}(x_0, i_0, u_0, r_0) = u_0^*$

If t is the current time, $u_t = u_{MPC}(x_t, i_t, u_t, r_t),\ i_{t+1} = i_t + (Cx_t - r)$

Note that the controller is dynamic, includes an integrator

If closed-loop is stable and the response converges to steady-state, then $i_t \to$ const, $Cx_t \to r$ (irrespective of model mismatch or constant disturbances)

Same basic critisisms apply. In addition, the extended state has higher dimension as it includes u_{k-1}

4.7 Reference Tracking Formulation 4

Incremental (rate-based) model

Another LQ-MPC based tracking formulation exploits directly the incremental (rate-based) model and is able to guarantee zero steady state offset if constraints are inactive in presence of constant additive disturbances

Define rates,

$$\Delta x_k = x_k - x_{k-1}, \quad \Delta u_k = u_k - u_{k-1}$$

Then

$$\Delta x_{k+1} = x_{k+1} - x_k = Ax_k + Bu_k - Ax_{k-1} - Bu_{k-1} = A\Delta x_k + B\Delta u_k$$

$$\begin{aligned} e_{k+1} &= Cx_{k+1} - r = C(x_{k+1} - x_k) + (Cx_k - r) = C\Delta x_{k+1} + e_k \\ &= CA\Delta x_k + CB\Delta u_k + e_k \end{aligned}$$

$$x_k = x_{k-1} + \Delta x_k$$

$$u_k = u_{k-1} + \Delta u_k$$

Incremental (rate-based) model

$$
\begin{cases}
\Delta x_{k+1} = A\Delta x_k + B\Delta u_k \\
e_{k+1} = CA\Delta x_k + CB\Delta u_k + e_k \\
x_k = x_{k-1} + \Delta x_k \\
u_k = u_{k-1} + \Delta u_k
\end{cases}
$$

Reference tracking formulation 4

$$\underset{\Delta u_0, \Delta u_1, \cdots, \Delta u_{N-1}}{\text{Minimize}} \quad J_N = \sum_{k=0}^{N-1} e_k^{\mathrm{T}} Q_e e_k + \Delta u_k^{\mathrm{T}} R \Delta u_k$$

subject to

$$\Delta x_{k+1} = A\Delta x_k + B\Delta u_k$$

$$\Delta x_0 = x_0 - x_{-1} = \text{given}$$

$$e_0 = Cx_0 - r = \text{current tracking error}$$

$$x_0 = \text{ current state, } x_{-1} = \text{ previous state}$$

$$u_0 = \text{ current control, , } u_{-1} = \text{ previous control}$$

$$e_{k+1} = CA\Delta x_k + CB\Delta u_k + e_k$$

$$x_k = x_{k-1} + \Delta x_k, \ k = 1, \cdots, N,$$

$$u_k = u_{k-1} + \Delta u_k, \ k = 0, \cdots, N-1$$

$$x_{min} \leq x_k \leq x_{max}, \ k = 1, \cdots, N$$

$$u_{min} \leq u_k \leq u_{max}, \ k = 0, \cdots, N-1.$$

Reference tracking formulation 4

This tracking LQ-MPC problem can be re-written as the standard LQ-MPC problem for an extended system with a larger state vector,

$$x_k^{\text{ext}} = [\Delta x_k^{\mathrm{T}},\ e_k^{\mathrm{T}},\ x_{k-1}^{\mathrm{T}},\ u_{k-1}^{\mathrm{T}}]^{\mathrm{T}},$$

and the control input Δu_k. The extended state prediction model is given by

$$x_{k+1}^{\text{ext}} = \begin{bmatrix} A & 0 & 0 & 0 \\ CA & \mathbb{I}_{n_e \times n_e} & 0 & 0 \\ \mathbb{I}_{n_x \times n_x} & 0 & \mathbb{I}_{n_x \times n_x} & 0 \\ 0 & 0 & 0 & \mathbb{I}_{n_u \times n_u} \end{bmatrix} x_k^{\text{ext}} + \begin{bmatrix} B \\ CB \\ 0 \\ \mathbb{I}_{n_u \times n_u} \end{bmatrix} \Delta u_k$$

Remark: The states x_k, u_k in rate-based model are only included to enforce the constraints. If certain state or control channels are not constrained, the corresponding state or control variables can be removed from the rate-based model.

Reference tracking formulation 4

For the extended system, the state penalty matrix has the form,

$$Q^{\text{ext}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & Q_e & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The control penalty matrix is $R^{\text{ext}} = R$.

The cost becomes

$$J_N = \sum_{k=0}^{N-1} (x_k^{\text{ext}})^{\mathrm{T}} Q^{\text{ext}} x_k^{\text{ext}} + \Delta u_k^{\mathrm{T}} R^{\text{ext}} \Delta u_k$$

Reference tracking formulation 4

- Adding a terminal penalty to the cost may also be advantageous to ensure closed-loop stability.

- Unfortunately, the extended system may not be controllable due to augmentation of constrained variables as states. Hence DARE/DLQR may not be directly used to generate the terminal penalty.

- A terminal penalty on Δx_N and e_N only may, however, be defined based on the solution to DLQR/DARE corresponding to dynamics and input matrices of the form

$$\begin{bmatrix} A & 0 \\ CA & \mathbb{I}_{n_e \times n_e} \end{bmatrix}, \begin{bmatrix} B \\ CB \end{bmatrix},$$

and state and control weighting matrices selected by the designer, i.e.,

$$\begin{bmatrix} 0 & 0 \\ 0 & Q_e \end{bmatrix}, R,$$

to have the terminal cost consistent with the incremental cost. Let $\tilde{P}_\infty$ denote the solution to the corresponding DARE.

Reference tracking formulation 4

Then with the terminal penalty added, the cost function becomes

$$J_N = (x_N^{\text{ext}})^{\text{T}} P_N x_N^{\text{ext}} + \sum_{k=0}^{N-1} (x_k^{\text{ext}})^{\text{T}} Q^{\text{ext}} x_k^{ext} + \Delta u_k^{\text{T}} R^{\text{ext}} \Delta u_k$$

$$P_N = \begin{bmatrix} \tilde{P}_\infty & 0 \\ 0 & 0 \end{bmatrix}$$

Reference tracking formulation 4

The tracking LQ-MPC feedback law is defined by the first element of the optimal solution sequence that is

$$\Delta u_{MPC}(\Delta x_0,\ e_0,\ x_{-1},\ u_{-1}) = \Delta u_0^*.$$

When implementing MPC, this feedback law is applied to the current values of the state increment, $\Delta x_t = x_t - x_{t-1}$, tracking error, $e_t = Cx_t - r$, previous state, x_{t-1} and previous control u_{t-1}, i.e., the controller has the form,

$$u_t = u_{t-1} + \Delta u_{MPC}(\Delta x_t, e_t, x_{t-1}, u_{t-1}).$$

Note that this controller includes integral action at the output. The integral action helps ensure zero steady-state offset in tracking constant commands. It also ensures zero steady-state offset if constant unmeasured disturbances are acting on the right hand side.

Reference tracking formulation 4

Remark 1: If the problem is unconstrained (or constraints are inactive locally near the set-point), as $N \to \infty$, the MPC feedback law is expected to approach the LQR feedback law (assuming LQR solution exists) and be stabilizing for the model in the incremental form (with states $[\Delta x^{\mathrm{T}} \quad e^{\mathrm{T}}]^{\mathrm{T}}$ and control Δu), thereby guaranteeing $e_k \to 0$ as $k \to \infty$.

Remark 2: Suppose the terminal penalty, selected based on the above procedure, is used. Then if the constraints are inactive near the set-point, the MPC feedback law coincides with LQR feedback law for the model in the incremental form for any N and hence $e_k \to 0$ as $k \to \infty$.

Reference tracking formulation 4

If the system is affected by a constant disturbance, $d_k = d \; \forall k$, and the model has the form,

$$x_{k+1} = Ax_k + Bu_k + d_k,$$

the model in the incremental form

$$\Delta x_{k+1} = Ax_k + Bu_k + d_k - Ax_{k-1} - Bu_{k-1} - d_{k-1} = A\Delta x_k + B\Delta u_k$$

$$e_{k+1} = e_k + CA\Delta x_k + CB\Delta u_k$$

will remain the same as

$$\Delta d_k = d_k - d_{k-1} = 0$$

Consequently, the controller, at least in the unconstrained case, will reject constant disturbances d_k with zero steady-state offset ($y_k \to r$ as $k \to \infty$ irrespective of $d_k = d$).

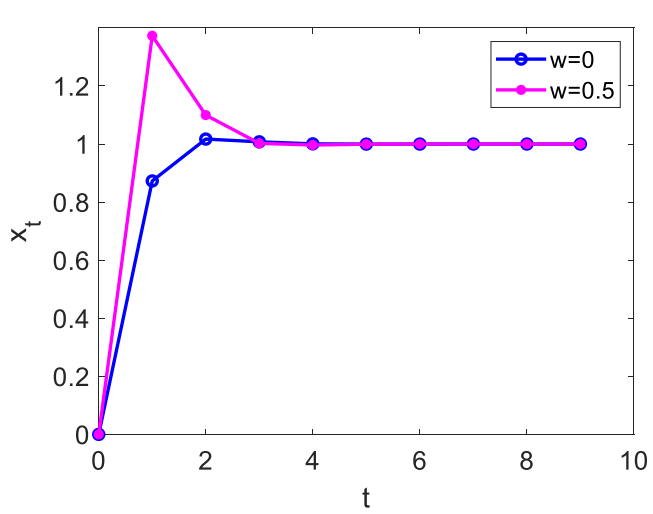
Reference tracking formulation 4

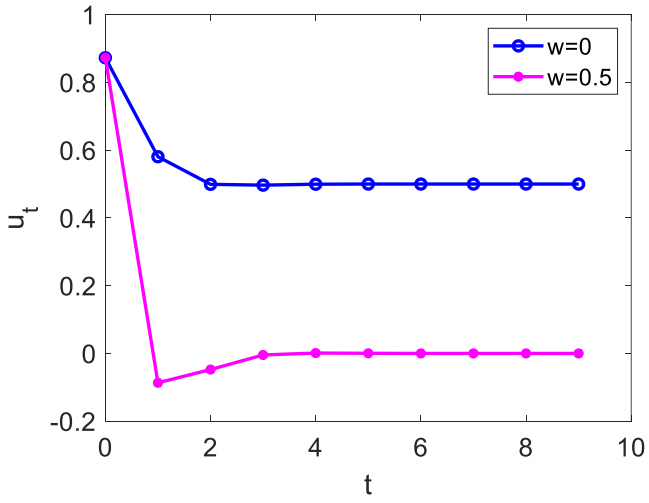
Back to our example...

$$A = \alpha = 0.5,\ B = \beta = 1.0,\ C = 1,\ r = 1,\ Q_e = 1,\ R = 0.1$$

$$N = 4$$

Simulate: $x_{t+1} = Ax_t + Bu_t + d_t,\ u_t = u_{t-1} + \Delta u_{MPC}(\Delta x_t, e_t, x_{t-1}, u_{t-1}),\ r_t = 1,\ d_t = w$



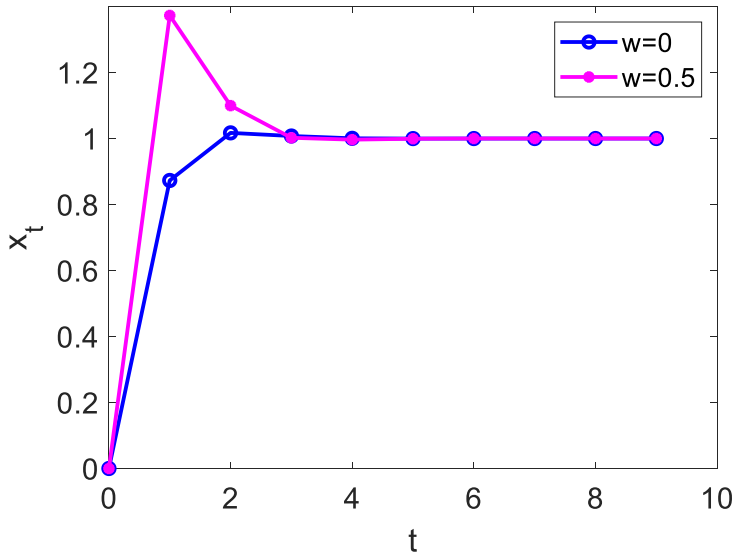


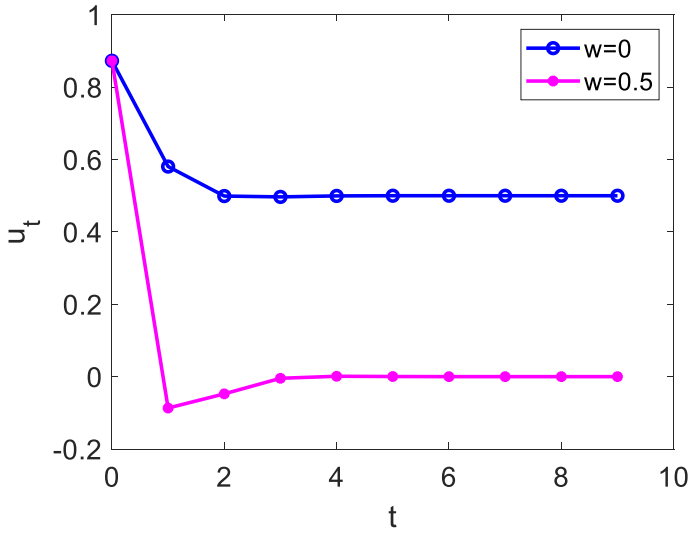
Note that tracking of a constant command occurs with zero offset in steady-state!

Reference tracking formulation 4

Change the horizon to $N = 1$ and add LQR terminal penalty with P_N chosen as described above.

By Bellman's principle of optimality, the solution must be the same as of infinite horizon P-LQ





4.8 Matlab Toolboxes

Matlab software

Multi-parametric Toolbox 3 for Matlab

http://people.ee.ethz.ch/~mpt/3/

Hybrid MPC Toolbox for Matlab

http://cse.lab.imtlucca.it/~bemporad/hybrid/toolbox/

Double integrator example

The model is given by

$$\dot{x}_1 = x_2$$
$$\dot{x}_2 = u + w$$

The sampling period is $T_s = 0.1$ sec. The constraints are given by

$$-1 \leq x_1 \leq 1.1, \quad -0.2 \leq x_2 \leq 0.2, \quad -0.1 \leq u \leq 0.1.$$

The objective is to achieve offset-free tracking of feasible (under constraints) desired position commands, i.e.,

$$x_1(t) \to r(t)$$

if the command $r(t)$ is constant, and the unmeasured disturbance w is constant.

MPT3 double integrator example

```
clear all
close all

% Continuous time model
    Ac = [0,1;0,0];
    Bc = [0;1];
    Cc = [1,0;0,1];
    Dc = [0;0];

% Discrete-time model
    Ts = 0.1;  % sampling period, sec
    sysd = c2d( ss(Ac, Bc, Cc, Dc), Ts );
    Ad = sysd.A;
    Bd = sysd.B;
    Cd = sysd.C;
    Dd = sysd.D;
```

MPT3 double integrator example

```
% Augmented model for tracking: states (x, u, r) and control du
    Ax = [Ad           Bd    0*Ad(:,1)
          0*Ad(1,:)     1    0
          0*Ad(1,:)     0    1      ] ; % states are x, u, r

    Bx = [Bd;1;0] ;                     % delta u  = new control

    Cx = [1,  0,      0,      0;
          0,  1,      0,      0;
          0,  0,      1,      0;
          0,  0,      0,      1];

    Dx = [0;0;0;0];

    Ex = [Cd(1,:) 0 -1] ;  % error = position - set-point

    model = LTISystem('A',Ax,'B',Bx,'C',Cx,'D',Dx, 'Ts', Ts);
```

MPT3 double integrator example

```
% Control constraints
    model.u.min= -1;
    model.u.max= -model.u.min;

% State constraints
    model.x.min=[-1, -0.2, -0.1, -1];
    model.x.max=[1.1, 0.2,  0.1,  1];

% Cost function
    model.u.penalty = QuadFunction(diag([0.1]))
    model.x.penalty = QuadFunction(Ex'*Ex)

% Horizon
    N=12;

% Generate MPC Controller
    ctrl = MPCController(model,N)
```

MPT3 double integrator example

```
% Simulate closed loop response
    x0 = [0;0];
    u  = 0;
    r0 = 1;
    Nsim = 300;

    x = x0;
    data.X = zeros(length(x0),Nsim);
    data.U = zeros(size(Bd,2),Nsim);
    data.R = zeros(1,Nsim);
    data.W = zeros(1,Nsim);
```

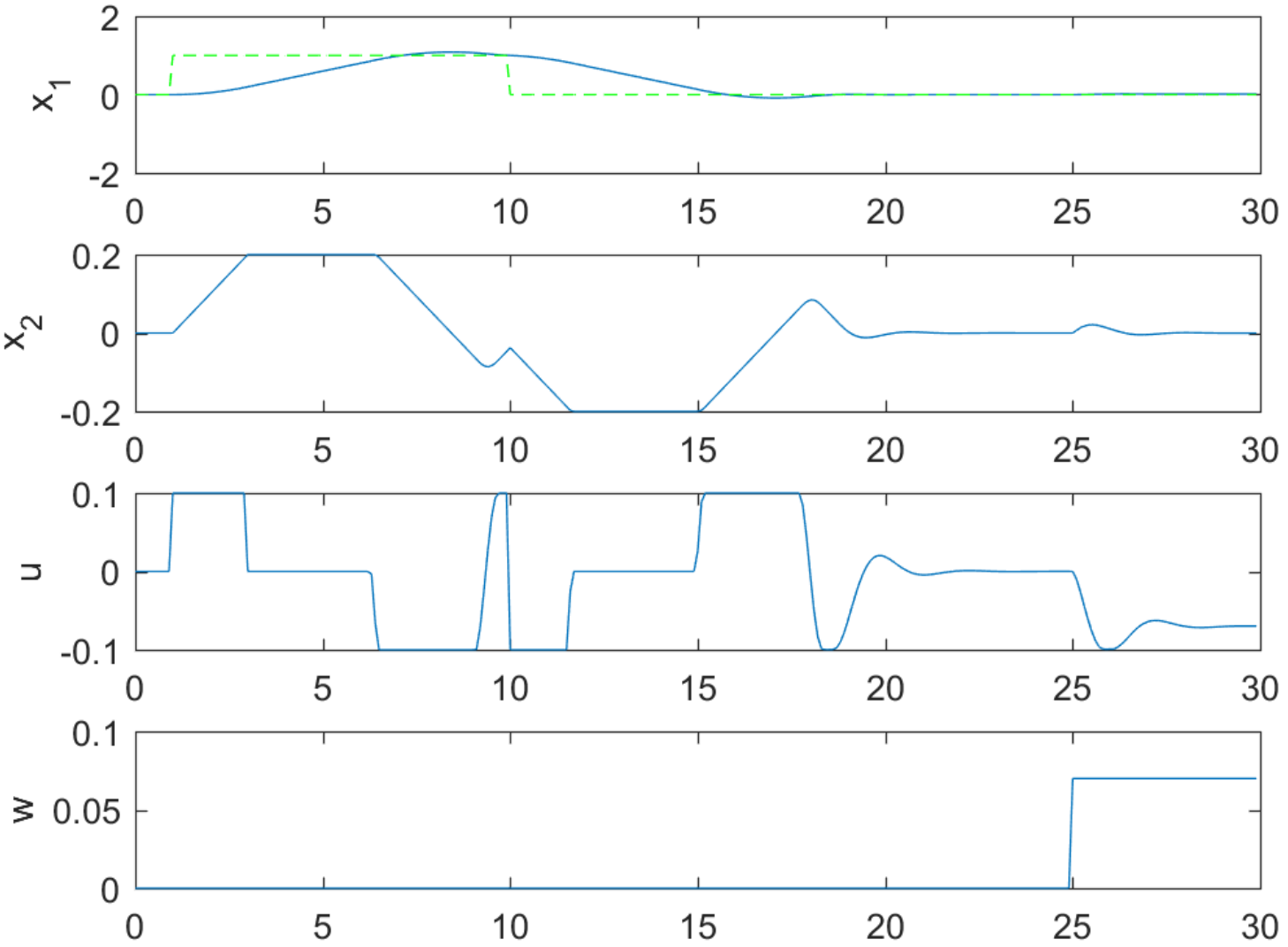
```
for i=1:Nsim,
        if (i-1)*Ts<1,
            r=0; w=0;
        elseif (i-1)*Ts<10,
            r = r0; w = 0.0;
        elseif (i-1)*Ts<25,
            r = 0;
        else,
            r = 0;
            w = 0.07;
        end;

        try,
            [du,feasible,openloop] = ctrl.evaluate([x(:); u; r]);
        catch,
            du = 0;
        end
        u = u + du;
        data.X(:,i)=x;
        data.U(:,i)=u;
        data.R(:,i)=r;
        data.W(:,i)=w;
        x = Ad*x + Bd*(u+w);
end;
```

MPT3 double integrator example

```
% Plot the responses
    figure;
    subplot(411)
    plot(([1:Nsim]-1)*Ts, data.X(1,:), ([1:Nsim]-1)*Ts,…
                data.R, 'g--');
    ylabel('x_1')
    subplot(412)
    plot(([1:Nsim]-1)*Ts, data.X(2,:))
    ylabel('x_2')
    subplot(413)
    plot(([1:Nsim]-1)*Ts, data.U(1,:))
    ylabel('u')
    subplot(414)
    plot(([1:Nsim]-1)*Ts, data.W(1,:))
    ylabel('w')
```

MPT3 double integrator example



Hybrid MPC toolbox for Matlab

```
addpath(genpath(<path to hybrid_tbx-win directory>))

clear variables
```

Replace with the directory path to your directory where hybrid toolbox is installed

Hybrid MPC toolbox for Matlab

```
% Define the linear discrete-time model from input to constrained
output
    Ts    = 0.1; % Sampling time
    model = c2d(ss([0 1;0 0],[0;1],[1,0;0,1],[0;0]),Ts,'zoh');

% Augmented model for tracking. States: (x,u,r). Control: du
    Ad = model.A;
    Bd = model.B;
    Cd = model.C;
    Ax = [Ad           Bd  0*Ad(:,1)
          0*Ad(1,:)    1    0
          0*Ad(1,:)    0    1     ] ; % states are x, u, r

    Bx = [Bd;1;0] ;                    % new control is delta u
    Cx = [1,  0,     0,     0;
          0,  1,     0,     0;
          0,  0,     1,     0;
          0,  0,     0,     1];

    Dx = [0;0;0;0]; % constrain state u and not actual u
    Ex = [Cd(1,:) 0 -1] ;          % error = position - set-point
    sysd   = ss(Ax, Bx, Cx, Dx, Ts) ;
```

Hybrid MPC toolbox for Matlab

```
% Define cost, constraints and horizon
    clear cost constraints horizon
    Qy = 1;
    cost.Q = Ex'*Qy*Ex ;
    cost.R = 0.1 ;
    cost.P = cost.Q ;
    cost.K = Bx'*0 ;
    Smpc   = ss(Ax,Bx,Cx,Dx,Ts) ;
    cost.rho = 0.1*Inf; % set to Inf for hard constraints

    horizon.N   = 12; % output horizon   \sum_{k=0}^{Ny-1}
    horizon.Nu  = 12; % input horizon    u(0),...,u(Nu-1)
    horizon.Ncu = 11; % input constraints horizon k=0,...,Ncu
    horizon.Ncy = 11; % output constraints horizon k=0,...,Ncy

    constraints.umax =  Inf ;
    constraints.umin = -constraints.umax ;

    constraints.ymax =  [1.1 0.2 0.1 1.1] ; % x, u, r
    constraints.ymin = -constraints.ymax ;

MPCctrl=lincon(sysd,'reg',cost,horizon,constraints,'qpact',0) ;
```

Hybrid MPC toolbox for Matlab

```
% Simulate closed-loop response
    x0 = [0;0];
    u  = 0;
    r0 = 1;
    Nsim = 300;

    x = x0;
    data.X = zeros(length(x0), Nsim);
    data.U = zeros(size(Bd,2), Nsim);
    data.R = zeros(1, Nsim);
    data.W = zeros(1, Nsim);
```

Hybrid MPC toolbox for Matlab

```
for i=1:Nsim,
        if (i-1)*Ts<1,
            r=0; w=0;
        elseif (i-1)*Ts<10,
            r = r0; w = 0.0;
        elseif (i-1)*Ts<25,
            r = 0;
        else,
            r = 0;
            w = 0.07;
        end;
        try,
            du = eval(MPCctrl,([x(:);u;r]));
        catch,
            du = 0;
        end
        u = u + du;
        data.X(:,i) = x;
        data.U(:,i) = u;
        data.R(:,i) = r;
        data.W(:,i) = w;
        x = Ad*x + Bd*(u + w);
end;
```

Hybrid MPC toolbox for Matlab

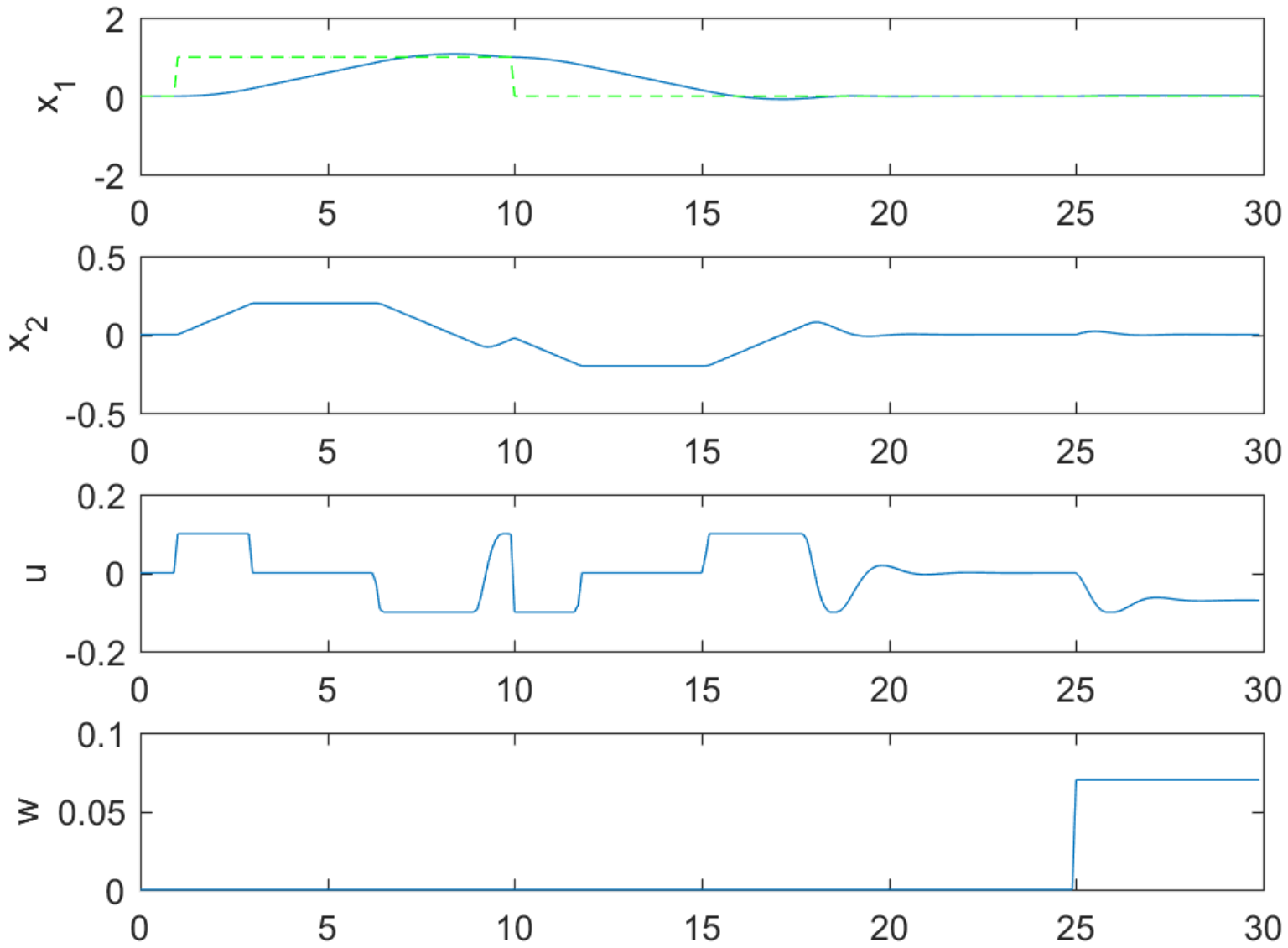
```
% Plot the responses
    figure;
    subplot(411)
    plot(([1:Nsim]-1)*Ts, data.X(1,:), ([1:Nsim]-1)*Ts,…
              data.R, 'g--');
    ylabel('x_1')
    subplot(412)
    plot(([1:Nsim]-1)*Ts, data.X(2,:))
    ylabel('x_2')
    subplot(413)
    plot(([1:Nsim]-1)*Ts, data.U(1,:))
    ylabel('u')
    subplot(414)
    plot(([1:Nsim]-1)*Ts, data.W(1,:))
    ylabel('w')

rmpath(genpath(<path to hybrid_tbx-win directory>))

return
```

Replace with the directory path to your directory where hybrid Toolbox is installed

Hybrid MPC toolbox for Matlab



4.9 Handling Infeasible Constraints Using Slack Variables

Handling infeasible constraints using slack variables

- State constraints can become infeasible during the system operation, i.e., no solution to MPC problem may exist that satisfies constraints.

- If state constraints are soft (slight violations are undesirable but do not lead to catastrophic failures), the system operation can be continued while the controller aggressively reduces constraint violation.

- In MPC the mechanism to handle potential infeasibility is through the slack variables.

- Mechanisms to ensure recursive feasibility will be discussed later in the course. However, in practical situations large disturbances or model mismatch can frequently cause infeasibility or constraint violation.

- Thus practical MPC controllers often use slack variables for state constraints while imposing tighter constraints than physical limits of the system.

Handling infeasible constraints using slack variables

Relax state constraints by introducing slack variables:

$$\begin{cases} x \leq x_{max} \\ -x \leq -x_{min} \end{cases} \Rightarrow \begin{cases} x \leq x_{max} + \epsilon \\ -x \leq -x_{min} + \epsilon \end{cases}$$

where ϵ is added as a manipulatable variable to the optimization problem.

Remark 1: Various modifications are possible, e.g., the slack variable can be a scalar, separate slacks can be used on the min and max side, only some constraints could be relaxed, etc.

Remark 2: The control constraints are typically not relaxed and treated as hard. Note that feasibility is trivially enforceable for the control constraints.

Modified LQ-MPC problem

$$\underset{u_0,u_1,\cdots,u_{N-1},\epsilon}{\text{Minimize}} \quad J_N = \sum_{k=0}^{N-1} x_k^{\mathrm{T}} Q x_k + u_k^{\mathrm{T}} R u_k + x_N^{\mathrm{T}} P x_N + \mu \epsilon^{\mathrm{T}} \epsilon$$

$$\begin{aligned}
\text{subject to} \quad & x_{k+1} = Ax_k + Bu_k \\
& x_0 = \text{(given current state)} \\
& x_{min} - \epsilon \leq x_k \leq x_{max} + \epsilon, \;\; k = 0, \cdots, N \\
& u_{min} \leq u_k \leq u_{max}, \;\; k = 0, \cdots, N-1
\end{aligned}$$

The MPC feedback law is defined by the first optimal move, u_0^*, i.e, $u_{MPC}^*(x_0) = u_0^*$.

The penalty weight $\mu > 0$ is large.

Remark: There are advantages to using nonsmooth penalties, $\mu||\epsilon||_\infty$ or $\mu||\epsilon||_1$ instead of $\mu\epsilon^{\mathrm{T}}\epsilon$. Specifically, for all μ sufficiently large, a constraint feasible solution will be generated if such a solution exist. This so called exact penalization approach. The problem is still solvable using QP as discussed later in the module.

Conversion to QP

Recall that to convert LQ-MPC problem to QP, we constructed the dependencies $X = SU + Mx_0$, $GU \leq W + Tx_0$, etc.

Replace our decision vector U by U^s, where

$$U^s = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \\ \epsilon \end{bmatrix},$$

where ϵ is augmented to U.

Replace $\bar{R}$ by $\bar{R}^s$ which includes the penalty on the slack variable

$$\bar{R}^s = \begin{bmatrix} R & \cdots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \cdots & R & 0 \\ 0 & \cdots & \cdots & \mu \mathbb{I}_{n_x \times n_x} \end{bmatrix}.$$

Conversion to QP

Replace S by

$$S^s = \begin{bmatrix} S & 0_{n_X \times n_\epsilon} \end{bmatrix}$$

Replace G by

$$G^s = \begin{bmatrix} S & \begin{bmatrix} -\mathbb{I}_{n_x \times n_\epsilon} \\ \vdots \\ -\mathbb{I}_{n_x \times n_\epsilon} \end{bmatrix} \\ -S & \begin{bmatrix} -\mathbb{I}_{n_x \times n_\epsilon} \\ \vdots \\ -\mathbb{I}_{n_x \times n_\epsilon} \end{bmatrix} \\ \mathbb{I}_{n_U \times n_U} & 0_{n_U \times n_\epsilon} \\ -\mathbb{I}_{n_U \times n_U} & 0_{n_U \times n_\epsilon} \end{bmatrix}$$

Remark: Note we use the same number of slack variables as channels in x's, so $n_x = n_\epsilon$. The same slack variable vector is applied for all k.

Conversion to QP

$$\begin{aligned} \underset{U^s}{\text{Minimize}} \quad & J_N = (U^s)^{\mathrm{T}} H^s U^s + 2(q^s)^{\mathrm{T}} U^s \\ \text{subject to} \quad & G^s U^s \leq W + T x_0. \end{aligned}$$

$$H^s = (S^s)^{\mathrm{T}} \bar{Q} S^s + \bar{R}^s = (H^s)^{\mathrm{T}} \succ 0$$

$$q^s = (S^s)^{\mathrm{T}} \bar{Q} M x_0$$

$$\bar{Q} = \begin{bmatrix} Q & \cdots & & \cdots & 0 \\ 0 & Q & & & \\ \vdots & & \ddots & & \\ 0 & \cdots & & Q & \\ 0 & \cdots & & 0 & P \end{bmatrix}$$

MPC from QP

Once the optimization problem is solved, and the solution $U^{s*}(x_0)$ is obtained, the LQ-MPC feedback law is defined as

$$u_{MPC}(x_0) = \begin{bmatrix} \mathbb{I}_{n_u \times n_u} & 0 & \cdots & 0 & 0_{n_u \times n_\epsilon} \end{bmatrix} U^{s*}(x_0)$$

4.10 Handling Disturbance Preview

Model with a disturbance preview

Suppose the model over the prediction horizon is given by

$$x_{k+1} = Ax_k + Bu_k + B_w w_k,$$

where the time instant $k = 0$ is the current time and w_k is a disturbance input. Such an input can represent a load in an electrical power system.

Suppose we happen to have a forecast of the future values of this input over some time window (this is called preview time window), i.e., $w_0,\ w_1,\ \cdots,\ w_{N_d}$ are known.

We then assume that $w_k = w_{N_d}$ for $k > N_d$, i.e., for $k = N_d{+}1, N_d{+}2, \cdots, N{-}1$.

Auxiliary model

To incorporate such a disturbance preview in MPC design we augment an auxilliary model for the disturbance in the form of a shift register,

$$\tilde{w}_{k+1} = A_w \tilde{w}_k,$$

where

$$A_w = \begin{bmatrix} 0 & I & 0 & \cdots & 0 \\ 0 & 0 & I & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & I \\ 0 & 0 & 0 & 0 & I \end{bmatrix}, \quad \tilde{w}_0 = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{N_d} \end{bmatrix}.$$

Note that

$$\tilde{w}_1 = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_{N_d} \\ w_{N_d} \end{bmatrix}, \; \tilde{w}_2 = \begin{bmatrix} w_2 \\ w_3 \\ \vdots \\ w_{N_d} \\ w_{N_d} \end{bmatrix}, \text{ etc.}$$

Augmented system and MPC design

We then change the model to

$$x_{k+1} = Ax_k + Bu_k + \tilde{B}_w \tilde{w}_k, \quad \tilde{B}_w = \begin{bmatrix} B_w & 0 & \cdots 0 \end{bmatrix}.$$

For the augmented system,

$$x_{k+1} = Ax_k + Bu_k + \tilde{B}_w \tilde{w}_k,$$

$$\tilde{w}_{k+1} = A_w \tilde{w}_k$$

the MPC feedback law will be defined by the first move u_0 of the suitable defined optimal control problem.

Note that it will be a function of the disturbance preview,

$$u = u_{MPC}(x_0, \tilde{w}_0) = u_0^*.$$

Unmeasured but estimated disturbances

Suppose now that the disturbance, w, is not measured and its future values are unknown. The following approach is often used:

- Estimate the current value of the disturbance from output measurements. Frequently, this is done by assuming a state-space model for the disturbance and designing an observer to estimate it along with the system state.

- Assume that the disturbance is constant over the prediction horizon and set it equal to the current estimated value, i.e.,

$$w_0 = w_1 = \cdots = w_{N-1} = \hat{w}_0,$$

  where $\hat{w}_0$ is the current estimate of the disturbance.

- Thus the MPC feedback law will have the form,

$$u = u_{MPC}(x_0, \hat{w}_0)$$

4.11 Handling Input Delays

Model with an input delay

Suppose the model over the prediction horizon is given by

$$x_{k+1} = Ax_k + Bu_{k-\tau},$$

where $\tau \in \mathbb{Z}_{\geq 0}$ is the input delay, and the time instant $k = 0$ is the current time instant. The delay τ is assumed to be constant.

To incorporate the delay in MPC design one approach involves augmenting an auxilliary model for the delay in the form of,

$$\tilde{u}_{d,k+1} = A_{\text{del}}\tilde{u}_{d,k} + B_{del}u_k,$$

where

$$A_{\text{del}} = \begin{bmatrix} 0 & I & 0 & \cdots & 0 \\ 0 & 0 & I & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & I \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad B_{\text{del}} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ I \end{bmatrix}.$$

Model with an input delay

Note that

$$\tilde{u}_{d,k} = \begin{bmatrix} u_{k-\tau} \\ u_{k-\tau+1} \\ \vdots \\ u_{k-1} \end{bmatrix}$$

We then change the model to

$$x_{k+1} = Ax_k + \tilde{B}\tilde{u}_{d,k} \quad \tilde{B} = \begin{bmatrix} B & 0 & \cdots & 0 \end{bmatrix}.$$

Consider the augmented system,

$$x_{k+1} = Ax_k + \tilde{B}\tilde{u}_{d,k},$$

$$\tilde{u}_{d,k+1} = A_{\text{del}}u_{d,k} + B_{\text{del}}u_k$$

MPC design for augmented system

For the augmented system,

$$x_{k+1} = Ax_k + \tilde{B}\tilde{u}_{d,k},$$

$$\tilde{u}_{d,k+1} = A_{\text{del}}u_{d,k} + B_{\text{del}}u_k$$

the MPC feedback law will be defined by the first move u_0 of the suitable defined optimal control problem.

Note that it will be a function of the current state and past values of the input,

$$u = u_{MPC}(x_0, u_{d,0}) = u_0^*, \quad \tilde{u}_{d,0} = \begin{bmatrix} u_{-1} \\ u_{-2} \\ \vdots \\ u_{-\tau} \end{bmatrix}$$

4.12 Beyond Quadratic - Treating “Linear Stage Costs”

Handling linear stage cost: Basic idea

Consider minimizing a function,

$$f(x,u) = \|Qx\|_p + \|Ru\|_p \to \min_{x,u}$$

where $p \in \{1, \infty\}$.

Note that

$$\|x\|_\infty = \max_{i=1,\cdots,n_x} |[x]_i|, \quad \|x\|_1 = \sum_{i=1}^{n_x} |[x]_i|,$$

$$Q \in \mathbb{R}^{n_x \times n_x}, \quad R \in \mathbb{R}^{n_u \times n_u}.$$

Handling linear stage cost: Basic idea

It is possible to re-write the above optimization problem in terms of minimizing a linear function subject to linear (affine) inequality constraints, specifically, as

$$c_x^{\mathrm{T}}\epsilon_x + c_u^{\mathrm{T}}\epsilon_u \to \min$$

subject to

$$\begin{bmatrix} Qx \\ -Qx \end{bmatrix} \leq W_x\epsilon_x, \quad \begin{bmatrix} Ru \\ -Ru \end{bmatrix} \leq W_u\epsilon_u,$$

where for $p = \infty$,

$$W_x = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}, \; c_x = 1, \; \epsilon_x \in \mathbb{R}, \; W_u = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}, \; c_u = 1, \; \epsilon_u \in \mathbb{R},$$

and for $p = 1$,

$$c_x^{\mathrm{T}} = \begin{bmatrix} 1, \cdots, 1 \end{bmatrix}, \; W_x = \begin{bmatrix} I \\ -I \end{bmatrix}, \epsilon_x \in \mathbb{R}^{n_x}, \; c_u^{\mathrm{T}} = \begin{bmatrix} 1, \cdots, 1 \end{bmatrix}, \; W_u = \begin{bmatrix} I \\ -I \end{bmatrix}, \epsilon_u \in \mathbb{R}^{n_u}.$$

MPC with linear stage costs

Let $p \in \{1, \infty\}$. Consider an optimal control problem,

$$\begin{aligned}
\underset{u_0, u_1, \cdots, u_{N-1}}{\text{Minimize}} \quad & J_N = \sum_{k=0}^{N-1} \|Qx_k\|_p + \|Ru_k\|_p + \|Px_N\|_p \\
\text{subject to} \quad & x_{k+1} = Ax_k + Bu_k \\
& x_0 = \text{current state} \\
& x_{min} \leq x_k \leq x_{max}, \;\; k = 1, \cdots, N \\
& u_{min} \leq u_k \leq u_{max}, \;\; k = 0, \cdots, N-1
\end{aligned}$$

By following the approach of transforming the norms into linear functions defined in terms of auxilliary variables, it can be shown that the above problem reduces to a Linear Programming (LP) problem.

The MPC law can be defined by the first move of the solution as in LQ-MPC case.

MPC with linear stage costs

Consider, for instance, the case $p = \infty$. Introduce auxilliary variables,

$$\epsilon_k^x \geq Q^i x_k, \ i = 1, \cdots, n_x,$$

$$\epsilon_k^x \geq -Q^i x_k, \ i = 1, \cdots, n_x,$$

$$\epsilon_k^u \geq -R^j u_k, \ j = 1, \cdots, n_u,$$

$$\epsilon_k^u \geq R^j u_k, \ j = 1, \cdots, n_u,$$

where Q^i denotes the ith row of Q and R^j denotes the jth row of R.

$$U = \begin{bmatrix} \epsilon_1^x \\ \vdots \\ \epsilon_N^x \\ \epsilon_0^u \\ \vdots \\ \epsilon_{N-1}^u \\ u_0 \\ \vdots \\ u_{N-1} \end{bmatrix} \Rightarrow$$

$$\begin{bmatrix} 1 & \dots & 1 & 0 & \dots & 0 \end{bmatrix} U \to \min_U$$

subject to

$$x_{min} \leq A^k x_0 + \sum_{i=0}^{k-1} A^i B u_{k-i-1} \leq x_{max}, \ k = 1, \cdots, N$$

$$u_{min} \leq u_k \leq u_{max}, \ k = 0, \cdots, N-1$$

$$\epsilon_k^x \geq Q^i x_k, \ \epsilon_k^x \geq -Q^i x_k, \ i = 1, \cdots, n_x,$$

$$\epsilon_k^u \geq -R^j u_k, \ \epsilon_k^u \geq R_j u_k, \ j = 1, \cdots, n_u,$$

$$\Updownarrow$$

$$GU \leq W + Sx_0$$

MPC based on LP versus MPC based on QP

- Small-signal response is usually less smooth with LP than with QP, because in LP an optimal point is typically on a vertex of a feasible region. Hence it may jump between vertices when parameters of the problem change
- When constraints dominate over performance (e.g., in transients), there is little difference between LP and QP solutions
- In some problems, linear stage cost is preferrable to quadratic cost. For instance, spacecraft fuel consumption is better reflected by 1-norm of the thrust.

4.13 State Estimation Using Moving Horizon Observers

Moving horizon estimation

A related to LQ-MPC problem on the observer side is that of Moving Horizon Estimation (MHE).

In its simplest form, one is given measurements, $y_0, \cdots, y_N \in \mathbb{R}^{n_y}$, a model, $x_{k+1} = Ax_k$, $y_k = Cx_k$, $x_k \in \mathbb{R}^{n_x}$, which may not perfectly fit the measurements, and has some a priori estimate of the initial state, $\bar{x}_0$.

The objective is to estimate the state sequence $x_0, x_1, \cdots, x_N$ that best fits the measurements by minimizing the following cost function,

$$J = (x_0 - \bar{x}_0)^{\mathrm{T}}(Q_0)^{-1}(x_0 - \bar{x}_0) + \sum_{k=0}^{N-1}(x_{k+1} - Ax_k)^{\mathrm{T}}Q^{-1}(x_{k+1} - Ax_k)$$
$$+ \sum_{k=0}^{N}(y_k - Cx_k)^{\mathrm{T}}R^{-1}(y_k - Cx_k).$$

The weighting matrices are symmetric and $Q_0 \succ 0$, $Q \succ 0$ and $R \succ 0$. The minimization is performed with respect to the unknown state sequence $x_0, \cdots, x_N$.

Moving horizon estimation

Note that no statistical information about the process noise and measurement noise is needed to formulate MHE (unlike Kalman filter).

However, further analysis suggests that if statistical information is available, Q is best set to the process noise covariance matrix, R is best set to measurement noise covariance matrix and Q_0 to the a priori state estimate covariance error. See Adaptive Kalman Filter literature on how to estimate these covariance matrices from data.

To solve the MHE problem, we proceed in the same manner as in LQ-MPC.

Stack the states and outputs into larger vectors,

$$X = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_N \end{bmatrix}, \quad Y = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_N \end{bmatrix}.$$

Moving horizon estimation

1. It is possible to write the cost J in the form,

$$J = (Y - \bar{C}X)^{\mathrm{T}} \bar{R}^{-1} (Y - \bar{C}X) + (LX - MX)^{\mathrm{T}} \bar{Q}^{-1} (LX - MX) + (\Gamma X - \bar{x}_0)^{\mathrm{T}} Q_0^{-1} (\Gamma X - \bar{x}_0),$$

   for appropriately defined Y, $\bar{C}$, $\bar{R}$, $\bar{Q}$, M, L, Γ (how?).

2. One can derive an expression for the best estimate, $\hat{X}$, which minimizes J with respect to X, as a function of Y and $\bar{x}_0$ (how?). This is called batch estimation.

3. In the theory of Moving Horizon Estimation, the following solution is derived by the application of forward dynamic programming (see the book by Rawlings, Mayne and Diehl),

$$\begin{aligned}
&\hat{x}_k^- = A\hat{x}_{k-1} && \text{(a priori state estimate update)}\\
&P_k^- = Q + AP_{k-1}A^{\mathrm{T}} && \text{(a priori covariance matrix update)}\\
&L_k = P_k^- C^{\mathrm{T}} (CP_k^- C^{\mathrm{T}} + R)^{-1} && \text{(gain)}\\
&\hat{x}_k = \hat{x}_k^- + L_k (y_k - C\hat{x}_k^-) && \text{(a posteriori state estimate update)}\\
&P_k = P_k^- - P_k^- C^{\mathrm{T}} (CP_k^- C^{\mathrm{T}} + R)^{-1} CP_k^- && \text{(a posteriori covariance matrix update)}\\
&(\hat{x}_0^-, P_0^-) = (\bar{x}_0, Q_0) && \text{(initialization)}
\end{aligned}$$

4. What are potential advantages of this solution over the batch method, e.g., from implementation perspective?

Moving horizon estimation

- Suppose the system has measured inputs, and the model is of the form,

$$x_k = Ax_{k-1} + Bu_{k-1}.$$

- Then the only change in MHO is in the a priori state estimate update (why?), which takes the form,

$$\hat{x}_k^- = A\hat{x}_{k-1} + Bu_{k-1}$$